

Ripassa

Documentazione di progetto

App personale per ripassi programmati con ripetizione temporale

Nikita Piraino

(codice interamente generato da AI, con supervisione)

7 luglio 2026

Indice

I	La soluzione, in parole semplici	2
1	Il problema	3
2	La soluzione: Ripassa	4
2.1	Cosa fa	4
2.2	Come si usa	4
2.3	Le decisioni di fondo	4
3	Cosa vogliamo realizzare	6
3.1	Stato attuale (fatto)	6
3.2	Prossimi passi	6
3.3	Possibili sviluppi futuri (oggi fuori scopo)	6
II	Documentazione tecnica	7
4	Stack e vincoli	8
5	Architettura	9
5.1	Mappa del codice	9
6	Modello dati	10
6.1	ripassi	10
6.2	occorrenze	10
6.3	allegati	11
6.4	cache_allegati (solo SQLite on-device)	11
7	Sincronizzazione cross-device	12
8	Cache locale e rotazione	13
9	Sicurezza	14
9.1	Autenticazione	14
9.2	Row Level Security	14
10	Gestione allegati	15
11	Test e qualità	16
12	Build e distribuzione	17
12.1	Fase 0 — sviluppo (Expo Go)	17
12.2	Fase 1 — app installate (vedi BUILD.md)	17

Parte I

La soluzione, in parole semplici

Capitolo 1

Il problema

Chi studia con il metodo della *ripetizione programmata* rivede lo stesso argomento a intervalli crescenti: il giorno dopo, dopo una settimana, dopo un mese, dopo sei mesi. Ogni “ripasso” ha bisogno del suo materiale: le foto degli appunti, i PDF delle dispense, qualche nota scritta al volo.

L’app usata finora per questo scopo (“Genio in 21 Giorni”) ha il concetto giusto ma un difetto strutturale: su iPad salva i dati su iCloud, su Android su Google Drive, e i due mondi non si parlano. Il risultato è che il materiale di studio è frammentato tra dispositivi, e quello che si aggiunge da un tablet non si ritrova sul telefono.

Capitolo 2

La soluzione: Ripassa

Ripassa è un'app personale, pensata per un solo utente, che fa una cosa sola e la fa su tutti i dispositivi insieme: **Android (Samsung), iPad e Mac vedono sempre gli stessi ripassi, le stesse date, gli stessi allegati**, grazie a un unico account e a un unico archivio centrale (Supabase).

2.1 Cosa fa

- **Crea un ripasso** con titolo e note libere.
- **Genera da sola le date di ripasso**: +1 giorno, +1 settimana, +1 mese, +6 mesi dal momento della creazione. Un interruttore (spento di default) aggiunge anche un ripasso “+1 ora” per il ripasso immediato.
- **Allega materiale**: foto scattate al momento, immagini dalla galleria, PDF e altri file. Gli allegati si possono rinominare, riordinare, eliminare.
- **Sincronizza in tempo reale**: una modifica fatta dal telefono compare sull'iPad senza fare nulla.
- **Funziona anche senza rete** (in treno, in metro): l'app tiene in locale gli allegati dei ripassi di *ieri, oggi e domani*, scaricandoli in anticipo e cancellando dal dispositivo quelli che non servono più. Il materiale resta comunque sempre al sicuro nel cloud.

2.2 Come si usa

L'app ha tre schermate, più il login:

1. **Accesso** — una volta sola per dispositivo, con l'account Google oppure con email e password. Da lì in poi la sessione resta attiva.
2. **Home** — l'elenco dei ripassi in due sezioni: quelli in programma (ordinati per data del prossimo ripasso) e lo storico. In alto la ricerca, in basso il pulsante per aggiungerne uno.
3. **Scheda ripasso** — titolo, note, l'elenco delle date programmate (ognuna si può segnare come completata, posticipare o anticipare) e l'accesso agli allegati.
4. **Allegati** — la galleria del materiale: si apre a schermo intero, si rinomina, si riordina, si elimina.

2.3 Le decisioni di fondo

- **Un solo account per tutto**, niente iCloud né Google Drive: il problema dell'app precedente non deve ripresentarsi.

- **Tutto gratuito** dove possibile; unica eccezione accettabile un costo minimo (~2 €/mese) se davvero necessario.
- **Niente statistiche, grafici o notifiche push:** fuori scopo, per scelta esplicita.
- **Codice scritto interamente da AI**, supervisionato ma non scritto a mano: lo stack è stato scelto anche per essere ben documentato e “generabile” in modo affidabile.
- **App nativa**, non sito web: serve accesso a fotocamera, file e archiviazione locale per l’uso offline.

Capitolo 3

Cosa vogliamo realizzare

3.1 Stato attuale (fatto)

L'MVP è completo e funzionante su Android via Expo Go: login (Google e email/password), creazione ripassi con date automatiche, allegati con compressione delle foto, sincronizzazione in tempo reale, cache offline con rotazione automatica, ricerca, storico. Il codice è coperto da una suite di test sulla logica di calcolo (34 test) e ha superato un audit di sicurezza (dettagli nella parte tecnica).

3.2 Prossimi passi

1. **App installate stabilmente** (“Fase 1”): un APK per il Samsung costruito nel cloud di Expo (gratis), e l'app per iPad firmata dal Mac con l'Apple ID gratuito. Con l'app installata il login persiste e non serve più Expo Go né la rete del Mac. La procedura completa è in BUILD.md.
2. **Validazione sul Mac M2**: verificare che l'app per iPad giri nativamente sul Mac come app “Designed for iPad” (rischio principale individuato dalla specifica).
3. **Decisione consapevole sul rinnovo settimanale**: con l'Apple ID gratuito la firma dell'app iPad scade ogni 7 giorni e va rinnovata ricollegando l'iPad al Mac. L'alternativa è l'Apple Developer Program (99 \$/anno). Si parte gratis e si valuta sull'uso reale.

3.3 Possibili sviluppi futuri (oggi fuori scopo)

Non fanno parte dell'MVP, ma l'architettura non li preclude: scrittura offline (creare ripassi senza rete, con coda di sincronizzazione), uso multi-utente (le protezioni dati per-utente sono già attive sul server, in previsione di proporre l'app a terzi), pubblicazione sugli store.

Parte II

Documentazione tecnica

Capitolo 4

Stack e vincoli

Livello	Scelta	Motivo
Frontend	React Native + Expo SDK 54	un codebase per Android/iPadOS/macOS
Backend	Supabase (Postgres, Storage, Realtime, Auth)	gratuito per uso personale, realtime nativo, un solo account
Cache locale	expo-sqlite + FileSystem	lettura offline senza servizi esterni
Autenticazione	Supabase Auth	Google OAuth (PKCE) + email/password
Test	jest-expo	logica pura testabile senza dispositivo

Limiti del piano gratuito Supabase (verificati, luglio 2026): 500 MB database, 1 GB storage file, 5 GB/mese di banda in uscita, pausa automatica del progetto dopo 7 giorni senza richieste. Il tetto di 1 GB è il vincolo più concreto: mitigato comprimendo le immagini lato client prima dell'upload (ridimensionamento a 1600px, JPEG 70%).

Capitolo 5

Architettura

Separazione **Model / Controller / View** obbligatoria:

```
View      (componenti React Native; tema colori isolato in theme.ts)
  | props, callback
Controller (custom hook React: stato + orchestrazione, nessun JSX)
  | funzioni pure / repository
Model     (client Supabase, query, cache SQLite, tipi TypeScript)
```

Regola pratica: *nessuna View importa il client Supabase o un repository del Model*; ogni accesso ai dati passa da un hook del Controller. La regola è stata verificata e ripristinata durante l'audit (l'apertura degli allegati era l'unica violazione, ora corretta).

5.1 Mappa del codice

File	Responsabilità
App.tsx	gating autenticazione, navigazione stack
src/config/supabase.ts	unica istanza del client; PKCE; refresh token via App-State
src/config/secureAuthStorage.ts	sessione cifrata in Keychain/Keystore (chunked)
src/theme/theme.ts	palette blu/giallo isolata
src/model/types.ts	tipi del dominio (specchiano lo schema Postgres)
src/model/occorrenzeDates.ts	calcolo puro delle date (+1h/+1g/+1s/+1m/+6m)
src/model/ripassiRepo.ts	CRUD ripassi/occorrenze su Supabase
src/model/allegatiRepo.ts	upload (compressione, base64), signed URL, riordino
src/model/localCache.ts	I/O cache: SQLite <code>cache_allegati</code> + filesystem
src/model/cacheLogic.ts	logica pura della rotazione (finestra, selezione)
src/model/fileUtils.ts	estensioni file, decodifica base64
src/controller/useAuth.ts	sessione, login Google (PKCE) / email, logout
src/controller/useRipassi.ts	stato ripassi, CRUD, subscription Realtime
src/controller/useLocalCache.ts	rotazione cache all'apertura (max 1/giorno)
src/controller/useAllegati.ts	picker, upload, apertura allegati
src/controller/RipassiContext.tsx	istanza unica condivisa (una sola subscription)
src/view/screens/*	Login, Home, FormRipasso, DettaglioAllegati
src/view/components/ui.tsx	Button, Card, Badge, SectionTitle
src/view/format.ts	date in italiano, etichette Oggi/Domani/Ieri
supabase/schema.sql	tabelle, trigger, RLS, Realtime, bucket (idempotente)
src/**/__tests__/*	34 test jest-expo sulla logica pura

Capitolo 6

Modello dati

Tre tabelle su Postgres (Supabase), tutte con RLS attiva, più una tabella solo locale sul dispositivo.

6.1 ripassi

Campo	Tipo	Note
id	uuid	PK, <code>gen_random_uuid()</code>
user_id	uuid	FK <code>auth.users</code> , default <code>auth.uid()</code>
titolo	text	
note	text	nullable, testo semplice
created_at / updated_at	timestampz	updated_at mantenuto da trigger

6.2 occorrenze

Campo	Tipo	Note
id	uuid	PK
ripasso_id	uuid	FK → ripassi, <code>ON DELETE CASCADE</code>
user_id	uuid	ridondante ma necessario per RLS efficiente
scheduled_at	timestampz	
is_manual_1h	boolean	true solo per l'occorrenza +1 ora
is_completed	boolean	default false
created_at / updated_at	timestampz	

Alla creazione di un ripasso vengono inserite le occorrenze a **+1 giorno**, **+1 settimana**, **+1 mese**, **+6 mesi** (`occorrenzeDates.ts`); l'occorrenza **+1 ora** solo se l'interruttore è attivo. Nota documentata: il 31 gennaio +1 mese trabocca su marzo (comportamento standard di JavaScript, coperto da test).

6.3 allegati

Campo	Tipo	Note
id	uuid	PK
ripasso_id	uuid	FK → ripassi, cascade
user_id	uuid	per RLS
display_name	text	modificabile dall'utente
original_file_name	text	nome originale sul dispositivo
storage_path	text	<user_id>/<ripasso_id>/<uuid>.<ext>
order_index	integer	riordino manuale
mime_type / size_bytes	text / bigint	

6.4 cache_allegati (solo SQLite on-device)

Campo	Tipo	Note
allegato_id	TEXT	PK
local_uri	TEXT	percorso nel filesystem del dispositivo
cached_at	TEXT	ultimo giorno (locale) in cui era in finestra

Capitolo 7

Sincronizzazione cross-device

- Ogni scrittura passa da Supabase; ogni dispositivo mantiene **una sola** subscription Realtime (in `RipassiContext`) sulle tre tabelle: alla ricezione di un evento il Controller ricarica e la View si ri-renderizza.
- Conflitti: *last-write-wins* su `updated_at` (trigger server-side). Sufficiente per un singolo utente su più dispositivi.
- Assunzione dichiarata: creare/modificare richiede connessione; la cache locale serve alla *lettura* offline. La scrittura offline (outbox) è fuori MVP.

Capitolo 8

Cache locale e rotazione

Finestra mantenuta in locale: **ieri, oggi, domani** — calcolata sui *giorni locali del dispositivo*, non UTC (un ripasso all'1:00 di notte conta per il giorno giusto).

Ad ogni apertura dell'app (al massimo una volta al giorno per sessione):

1. si scaricano gli allegati dei ripassi con almeno un'occorrenza in finestra (se già presenti si aggiorna solo `cached_at`);
2. si eliminano file e righe di cache degli allegati **non più appartenenti** alla finestra corrente — inclusi quelli eliminati da remoto.

Decisione rilevante emersa dall'audit: l'eliminazione avviene per *appartenenza alla finestra* e non per data di download (`cached_at`); il criterio originale cancellava e ricaricava di continuo i file ancora validi ma scaricati due o più giorni prima (regressione coperta da test). Il dato remoto non viene mai toccato dalla rotazione. Al logout la cache viene svuotata (i file appartengono all'utente).

Capitolo 9

Sicurezza

9.1 Autenticazione

- **OAuth Google con PKCE** (non implicit flow): il redirect `ripassa://` trasporta solo un `?code=` opaco, scambiato per la sessione con un *code verifier* noto solo al client. Un URL di redirect intercettato o loggato non è sufficiente a rubare la sessione.
- **Sessione cifrata a riposo**: JWT e refresh token in `expo-secure-store` (Keychain iOS / Keystore Android), con spezzettamento in voci da ≤ 2 KB (limite di SecureStore). Prima erano in AsyncStorage, in chiaro sul filesystem.
- **Refresh in foreground**: listener AppState che riavvia l'auto-refresh del token quando l'app torna attiva (pattern raccomandato Supabase); elimina i re-login forzati.
- Solo la chiave *publishable* è nel client; la chiave *secret* non compare da nessuna parte nel progetto.

9.2 Row Level Security

Tutte le tabelle hanno RLS attiva con isolamento per utente. Su `occorrenze` e `allegati` la policy verifica sia `user_id = auth.uid()` sulla riga, sia — difesa in profondità — che il ripasso genitore appartenga all'utente:

```
create policy occorrenze_owner on public.occorrenze
for all using (
  auth.uid() = user_id
  and exists (select 1 from public.ripassi r
              where r.id = ripasso_id and r.user_id = auth.uid())
) with check ( -- identico --);
```

Lo Storage usa un bucket privato `allegati` con policy che vincola l'accesso alla prima cartella del percorso (`<user_id>/...`); i download passano da URL firmati temporanei (1 ora).

Capitolo 10

Gestione allegati

- **Upload:** compressione immagini (1600px, JPEG 70%), lettura base64 e decodifica in bytes (decoder puro con lookup table, testato contro **Buffer**), upload su Storage, riga in tabella.
- **Apertura:** le immagini si mostrano in un viewer in-app a schermo intero (funziona con `file://` locali e URL firmati); i PDF in cache si aprono con il viewer di sistema (intent VIEW con `content://` su Android, share sheet su iOS); fallback: URL firmato nel browser. *Mai* un `file://` verso WebBrowser (su Android è una `FileUriExposedException`).
- **Eliminazione:** riga DB, file remoto e copia in cache locale.

Capitolo 11

Test e qualità

```
npm test          # 34 test jest-expo
npm run typecheck # tsc --noEmit (strict)
```

Coperti: generazione occorrenze (offset, flag +1h, fine mese), finestra cache in giorni locali (cambio mese/anno, mezzanotte), criterio di eliminazione per appartenenza (con test di regressione del bug originale), decodifica base64 (roundtrip contro **Buffer**, whitespace, binario), inferenza estensioni, formattazione date italiane. La logica è testabile perché estratta in moduli puri senza dipendenze native.

Capitolo 12

Build e distribuzione

12.1 Fase 0 — sviluppo (Expo Go)

`npm start` sul Mac, scansione QR con Expo Go (stessa rete Wi-Fi; il tunnel ngrok è attualmente inaffidabile).

12.2 Fase 1 — app installate (vedi BUILD.md)

- **Android:** `eas build -p android -profile preview` → APK dal cloud Expo (gratis, ~15 build/mese), installazione diretta. Variabili d'ambiente registrate su EAS (il `.env` locale non viene caricato).
- **iPad:** `npx expo prebuild -p ios + npx expo run:ios -device` con Apple ID gratuito (Personal Team). Limite: firma valida 7 giorni, rinnovo = rilanciare il comando con iPad collegato. Richiede Xcode e CocoaPods sul Mac.
- **Mac M2:** stessa app come “My Mac (Designed for iPad)” da Xcode. È il rischio 11.1 della specifica, in corso di validazione.
- Redirect OAuth per le build standalone: `ripassa://` va aggiunto agli URL consentiti in Supabase Auth.

Vincolo verificato: percorso senza spazi

La build iOS nativa è stata provata end-to-end (Xcode 26.6). Fallisce se il percorso del progetto contiene spazi — la cartella attuale `Side hustle` rompe lo script Expo `get-app-config-ios.sh`, invocato senza virgolette (`bash: .../Documents/Side: No such file or directory`). Spostando il progetto in un percorso senza spazi la stessa build raggiunge `** BUILD SUCCEEDED **` e produce `Ripassa.app` (binario universale arm64+x86_64). Il codice compila: l'unico ostacolo è lo spazio nel percorso. Android via cloud EAS non ne è affetto. Dettagli e comandi in BUILD.md.

Capitolo 13

Rischi aperti

1. Comportamento dei moduli nativi (fotocamera, picker) su macOS in modalità “Designed for iPad” — da testare sul dispositivo reale.
2. Consumo effettivo dello storage Supabase (1 GB) con l’uso reale — da misurare dopo le prime settimane.
3. Rinnovo settimanale della firma iPad con Apple ID gratuito — fastidio accettabile o no? Decisione dopo l’uso.